

# Despliegue de London Cafe CDJ en AWS + MongoDB Atlas

Guía paso a paso — Backend en EC2, base de datos en Atlas, HTTPS con dominio propio

**Ejemplo usado en esta guía:** el dominio `api.londoncafedj.com` es un placeholder. Sustitúyelo por tu dominio real en cada paso.

## 1. MongoDB Atlas

1. Entra a `cloud.mongodb.com` y crea un cluster gratuito (M0) si no tienes uno.
2. **Database Access** → crea un usuario, ej. `londoncafe_app`, con contraseña fuerte (guárdala).
3. **Network Access** → agrega `0.0.0.0/0` temporalmente (se cierra en el paso 8).
4. **Connect** → **Drivers** → copia el connection string:

```
mongodb+srv://londoncafe_app:<password>@cluster0.xxxxx.mongodb.net/london_cafe?retryWrites=t
```

## 2. Lanzar el EC2

1. Consola AWS → EC2 → **Launch Instance**: Ubuntu 24.04, tipo `t3.micro` (capa gratuita).
2. Crea o reutiliza un **Key Pair** (.pem) para SSH.
3. **Security Group**: abre `22` (solo tu IP), `80` y `443` (0.0.0.0/0). No abras el `4000` al público.
4. Asigna una **Elastic IP** a la instancia.
5. En tu DNS, crea un registro **A**: `api.tudominio.com` → Elastic IP.

## 3. Preparar el servidor

```
ssh -i tu-key.pem ubuntu@<ELASTIC_IP>
```

```
# Node.js LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs git nginx

# PM2 (mantiene el proceso vivo, reinicio automático)
sudo npm install -g pm2

# Clona el repo
git clone https://github.com/jose-esquivel-dev/TaskLondon_Dev.git
cd TaskLondon_Dev/backend
npm install --production
```

## 4. Variables de entorno (.env en el servidor)

```
cat > .env <<'EOF'
PORT=4000
MONGO_URI=mongodb+srv://londoncafe_app:TU_PASSWORD@cluster0.xxxxx.mongodb.net/london_cafe?re
JWT_SECRET=un-secreto-largo-y-aleatorio-distinto-al-de-dev
JWT_EXPIRES_IN=7d
EOF
```

Genera el `JWT_SECRET` con `openssl rand -hex 32`. No reuses el de desarrollo.

## 5. Levantar con PM2

```
pm2 start src/server.js --name london-cafe-api
pm2 save
pm2 startup # sigue las instrucciones que imprime, para autoarranque tras reboot
```

## 6. Nginx como proxy + HTTPS

```
sudo tee /etc/nginx/sites-available/london-cafe-api <<'EOF'
server {
    listen 80;
    server_name api.tudominio.com;
    client_max_body_size 20M;

    location / {
        proxy_pass http://localhost:4000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
```

```

        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

sudo ln -s /etc/nginx/sites-available/london-cafe-api /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

# HTTPS gratis
sudo apt-get install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.tudominio.com

```

## 7. Cargar los datos iniciales en Atlas

Desde tu computadora (no en el servidor):

```

cd backend
MONGO_URI="mongodb+srv://londoncafe_app:TU_PASSWORD@cluster0.xxxxx.mongodb.net/london_cafe?r

```

Usa la variable inline, no `npm run seed` (ese usaría tu `.env` local con `localhost`).

## 8. Cerrar el círculo de seguridad

- En Atlas → Network Access → elimina `0.0.0.0/0`, agrega solo la Elastic IP del EC2.
- En el Security Group del EC2, confirma que el puerto `4000` no esté expuesto públicamente — solo 80/443 vía Nginx.

## 9. Apuntar la app móvil a producción

En `mobile-app/app.json`, dentro de `expo.extra`:

```
"apiUrl": "https://api.tudominio.com"
```

## 10. Recompilar la app

```
cd mobile-app  
npx eas build --platform ios --profile preview
```

## Checklist resumen

| #  | Dónde       | Acción                                        |
|----|-------------|-----------------------------------------------|
| 1  | Atlas       | Cluster M0, usuario DB, IP allowlist temporal |
| 2  | AWS         | EC2 t3.micro + Elastic IP + Security Group    |
| 3  | DNS         | Registro A → Elastic IP                       |
| 4  | EC2         | Node, git clone, npm install, .env con Atlas  |
| 5  | EC2         | PM2 start + save + startup                    |
| 6  | EC2         | Nginx + Certbot HTTPS                         |
| 7  | Local       | Seed apuntando a Atlas                        |
| 8  | Atlas + AWS | Cerrar accesos abiertos                       |
| 9  | app.json    | apiUrl → dominio real                         |
| 10 | Mobile      | Rebuild con EAS                               |